



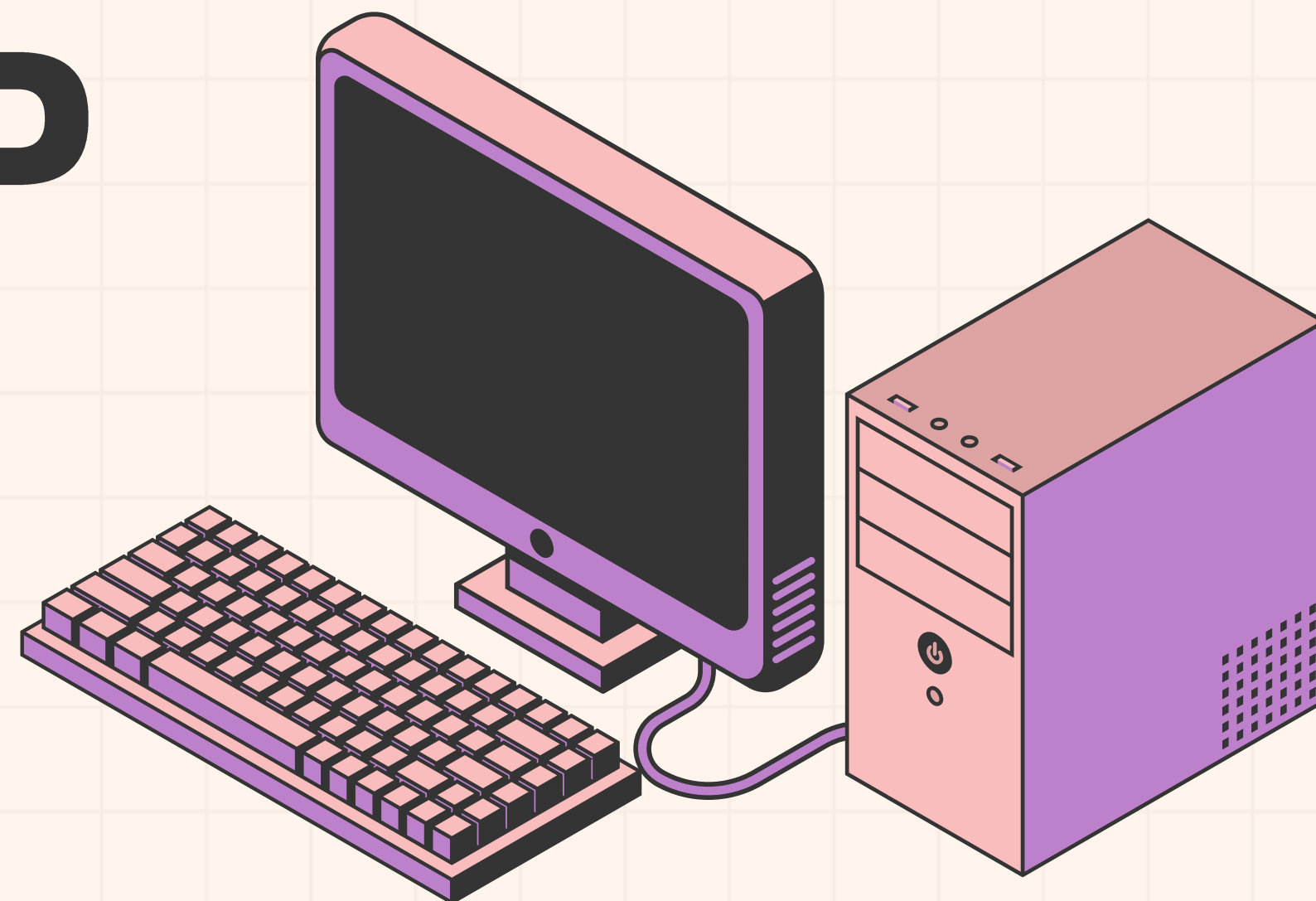
UNIVERSITÀ
DEGLI STUDI
FIRENZE

MARTINA
SCHIRONE

SHADOWMAP

COMPUTER GRAPHICS

In this project we implement, within a simple 3D scene, the **Shadow Mapping** technique for generating shadows.





UNIVERSITÀ
DEGLI STUDI
FIRENZE

MARTINA
SCHIRONE

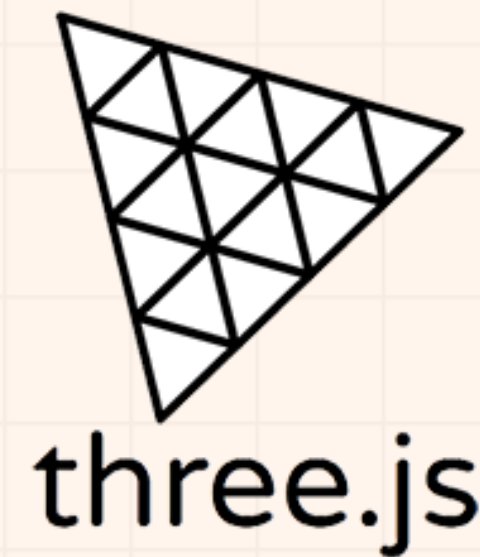
INTRODUCTION

This project implements the **Shadow Mapping** technique for real-time shadow generation in a 3D scene developed with **WebGL** and **Three.js**.

The scene consists of an **isometric room** containing two different light sources: a **Directional Light** and a **Point Light**.

In addition to traditional Shadow Mapping (Basic Shadow Mapping), the project allows you to compare different algorithms, including **Percentage Closer Filtering (PCF)** and **Variance Shadow Mapping (VSM)**.

From the UI, you can also modify parameters such as light **bias**, selectively activate or deactivate lights, graphic helpers, and finally display the **shadow depth map** using a toggle.





TECHNOLOGIES & RESOURCE USED

SOFTWARE AND FRAMEWORK

- **WebGL** – JavaScript graphic API
- **Three.js** – graphic library
- **JavaScript** – code language
- **Vite** – server build tool
- **WebStorm** – IDE

3D MODELS

- **Isometric Room** – <https://skfb.ly/6YEpW>
- **Cube** – created from code

IMPLEMENTED FUNCTIONALITIES

- **Directional Light**
- **Point Light**

- Interactively change the **position of lights**
- Visualization of the **lights helpers**.

- Adjust the **bias**
- Visualization of the **shadow map (depth texture)**

- **Basic Shadow Map**
- **PCF Shadow Map**
- **VSM Shadow Map**



CODE IMPLEMENTATION

Thanks to the three.js library the implementation of the shadowmap techniques is fairly simple.

Each JavaScript file is responsible for a specific part of the application:

- **lights.js** – defines and configures the lights in the scene
- **models.js** – loads the room model using **GLTFLoader** and creates the cube
- **scene.js** – initializes the renderer and the main scene
- **state.js** – contains all UI-related state variables used across the application
- **ui.js** – handles the creation and management of the user interface
- **shaders.js** – sets all the needed shaders for the shadow mapping
- **main.js** – core file that connects all components together and sets up the **OrbitControls** and **render loop**.

```
js lights.js  
js main.js  
js models.js  
js scene.js  
js shaders.js  
js state.js  
js stats.js  
style.css  
js ui.js
```

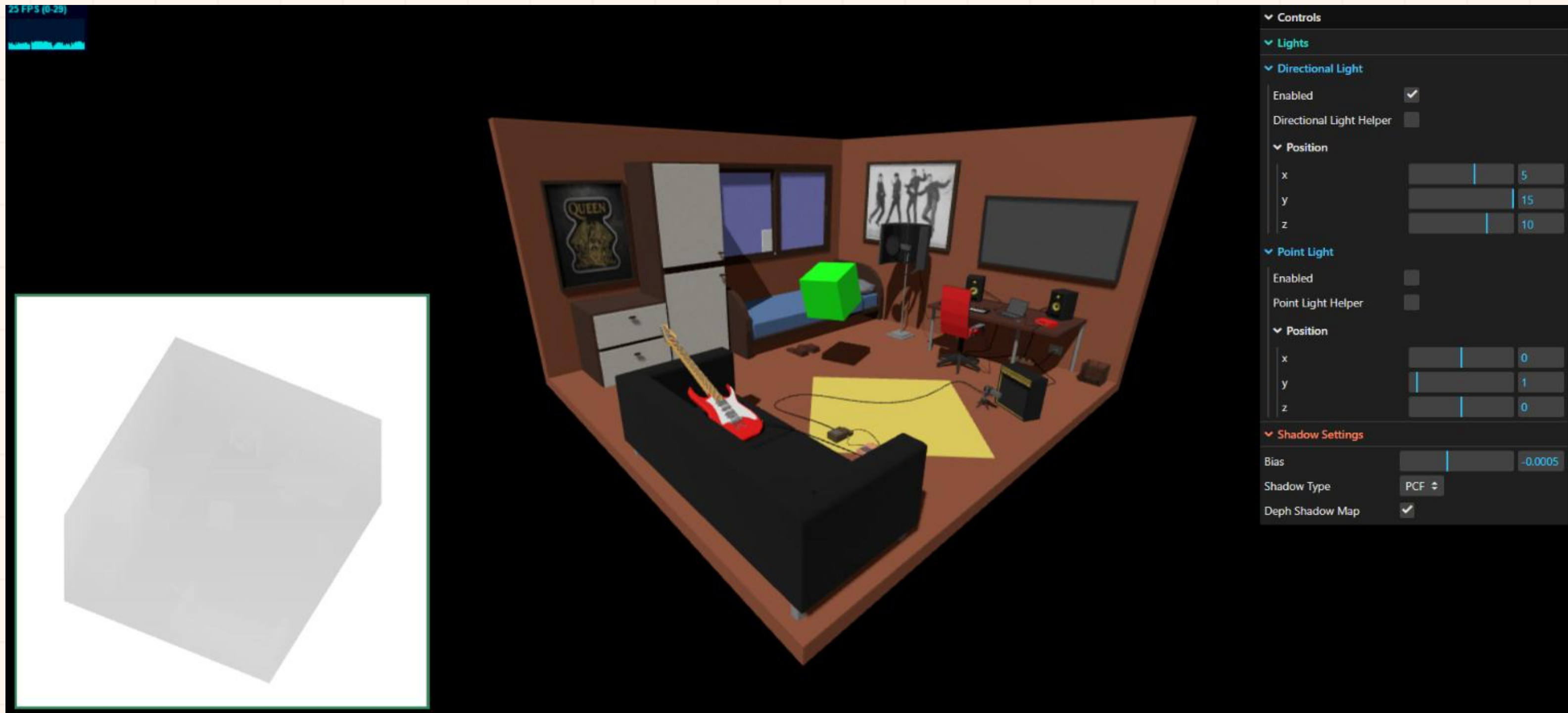
```
// 1° PASS: SHADOW MAP  
renderShadowMap();  
  
renderer.clear();  
  
// 2° PASS: MAIN SCENE  
renderer.render(scene, camera);
```



UNIVERSITÀ
DEGLI STUDI
FIRENZE

MARTINA
SCHIRONE

FULL PROJECT VISUAL





SHADOWMAPPING

WHAT IS IT?

The main idea is to determine which points in the scene are visible from the light source and which are hidden by other objects.

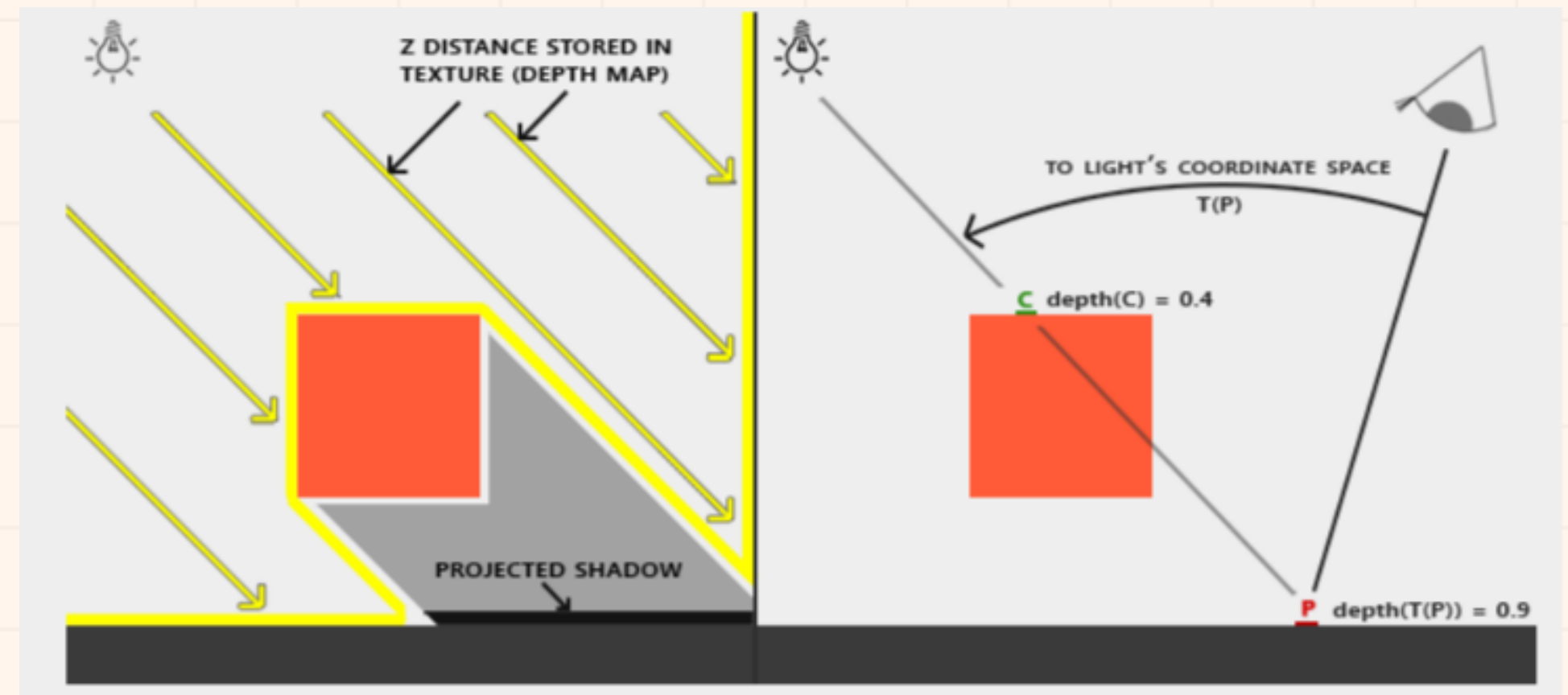
The rendering process occurs in two steps:

First rendering (from the light's perspective):

- We take a picture of the scene from the light source's perspective. The **distances of the visible objects** from the light are stored in a Depth Shadow Map (depth texture) using a **Frame Buffer Object (FBO)**.

Second rendering (from the camera's perspective):

- The scene is rendered normally. For each visible point in the scene, its **distance from the light is compared to the value stored in the Shadow Map**. If it is further away than the value recorded in the map texel, it means it is hidden by another object and is therefore in shadow.





SHADOWMAPPING TECHNIQUES

BASIC

TRADITIONAL SHADOW MAPPING

Performs a direct comparison between the fragment's depth and the depth stored in the shadow map texel.

Returns **0 or 1** depending on whether my fragment is in shadow or not.

Shadows generated with this technique have **jagged edges** and suffer from **shadow acne**.

PCF

PERCENTAGE CLOSE FILTERING

It performs **multiple shadow map samplings** around the texel corresponding to the fragment.

A depth comparison is performed for each sample and the **average** of the results is returned.

The shadows generated with this technique are **softer**, but it is **more expensive** than the basic technique.

VSM

VARIANT SHADOW MAPPING

Stores the first **two depth moments (mean and mean of squares)** in the shadow map.

This information is used during rendering to probabilistically estimate the presence of shadow in a fragment, using **mean and variance**.



BASIC SHADOW MAPPING

Let's take a closer look at how the shadow mapping shaders are coded.

Since there are two rendering passes, automatically four shaders are needed:

- **Vertex and Fragment Shaders from the 1° rendering,**
- **Vertex and Fragment Shaders from the 2° rendering.**

Shown here are the Vertex Shader and Fragment Shader from the 1° rendering, where the depth map is created.

```
vertexShader: `
    uniform mat4 lightMatrix;

    void main() {
        vec4 worldPos = modelMatrix * vec4(position, 1.0);
        gl_Position = lightMatrix * worldPos;
    }
`
```

```
fragmentShader: `
    #ifdef GL_ES
    precision highp float;
    #endif

    uniform bool useVSM;

    void main() {

        if (!useVSM) {
            // BASIC / PCF mode → only depth
            return;
        }

        // VSM: encode moments
        float depth = gl_FragCoord.z;

        float dx = dFdx(depth);
        float dy = dFdy(depth);
        float moment2 = depth * depth + 0.25 * (dx*dx + dy*dy);

        gl_FragColor = vec4(depth, moment2, 0.0, 1.0);
    }
`
```




```
shader.vertexShader = `
    uniform mat4 lightMatrix;
    varying vec4 vShadowCoord;
` + shader.vertexShader;

shader.vertexShader = shader.vertexShader.replace(
    {
        searchValue: `#include <begin_vertex>`,
        replaceValue: `
            #include <begin_vertex>

            vec4 worldPos = modelMatrix * vec4(position, 1.0);
            vShadowCoord = lightMatrix * worldPos;
        `
    },
    1
);
```

Here we can see the vertex shader used during the 2° rendering pass.

Since the scene uses a model created by another author, its original Material is preserved.

I extend the original vertex and fragment shaders using **onBeforeCompile**, a Three.js callback that is executed just before the shader program is compiled.

This allows **custom code** to be injected into the material original shaders.

This approach preserves Three.js' Physically Based Rendering (PBR) pipeline while adding only the computations required for custom shadow mapping.



Here is the Fragment Shader from the 2° render.

```
float getShadowBasic(vec2 uv, float depth) {  
    float closest = texture2D(shadowMapCustom, uv).r;  
    return step(closest + bias, depth);  
}
```

The fragment's position relative to the light is **normalized (divided by w)**, obtaining Normalized Device Coordinates (NDC).

These are then **converted** from the range $[-1,1]$ to $[0,1]$ to access the shadow map and compare depths.

We then check what type of shadow mapping we are implementing and call the relative shader function.

```
shader.fragmentShader = shader.fragmentShader.replace(  
    searchValue: '#include <dithering_fragment>',  
    replaceValue: '  
        vec3 proj = vShadowCoord.xyz / vShadowCoord.w;  
        proj = proj * 0.5 + 0.5;  
  
        float depth = proj.z;  
        float shadow;  
  
        if (useVSM) {  
            shadow = getVSMShadow(proj.xy, depth);  
        }  
        else if (usePCF) {  
            shadow = getShadowPCF(proj.xy, depth);  
        }  
        else {  
            shadow = getShadowBasic(proj.xy, depth);  
        }  
  
        diffuseColor.rgb *= (1.0 - shadow);  
  
        #include <dithering_fragment>  
    ');
```



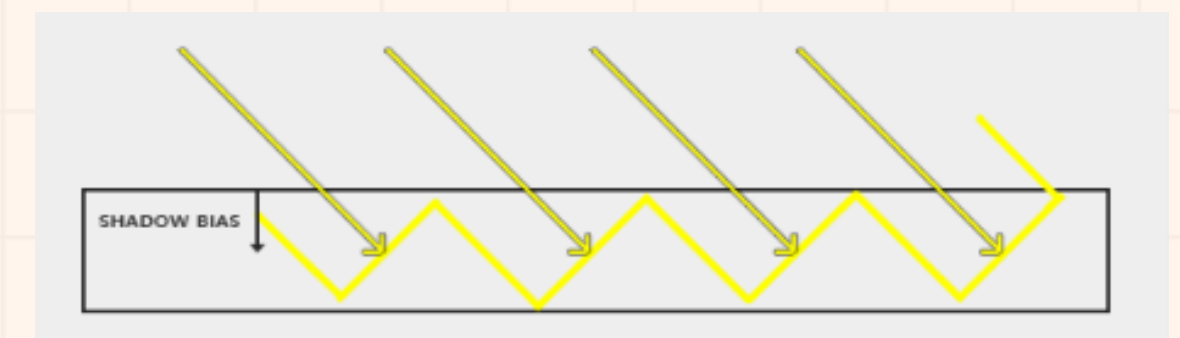
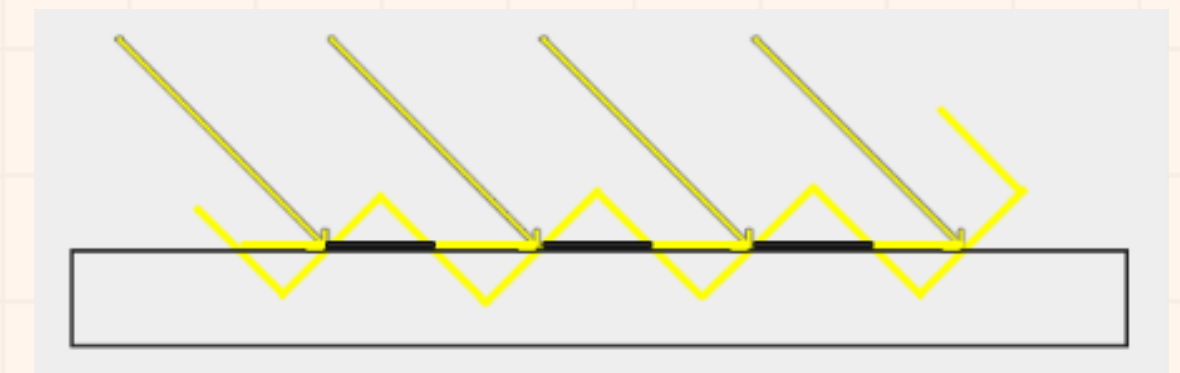
SHADOW ACNE & BIAS

One of the major problems with the basic shadow mapping technique, aside from the jagged rendering of shadows, is the so-called **shadow acne** phenomenon.

Shadow acne is an artifact caused by errors in depth comparisons, caused by the **limited resolution** of the shadow map, which causes multiple fragments to be compared to the same texel, and by the **angle of the light**.

One solution to this problem is to introduce a **bias**, a small tolerance in the depth comparison, to prevent illuminated fragments from being incorrectly classified as shadowed.

Excessive bias, however, can cause the phenomenon of **Peter Panning**, in which shadows appear visibly separated from the objects that cast them.

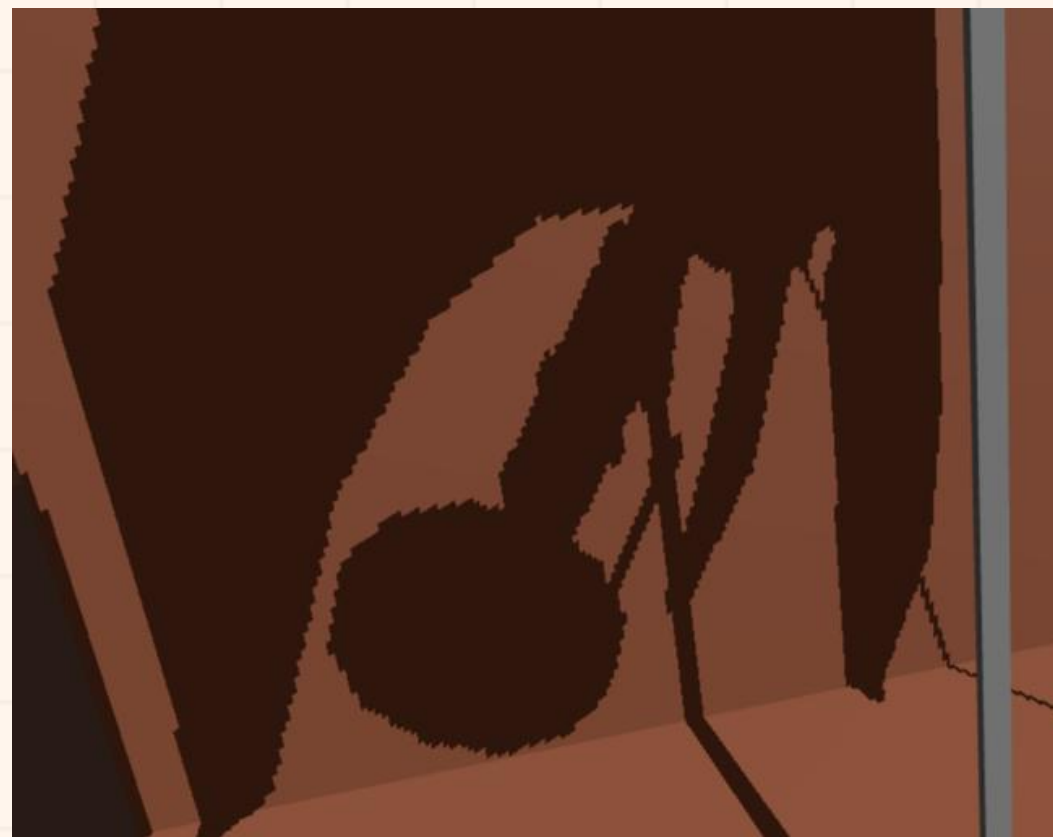




UNIVERSITÀ
DEGLI STUDI
FIRENZE

MARTINA
SCHIRONE

EXAMPLE OF SHADOW ACNE AND JAGGED SHADOWS





PCF SHADOW MAPPING

An **extension** of Basic Shadow Mapping.

Instead of performing a single comparison between the depth of the fragment and a texel of the shadow map, **multiple adjacent texels are sampled (3×3 grid)**. The depth test is performed for each sample, and the results are then **averaged** to obtain the final shadow value.

This approach produces softer, more realistic shadows, but introduces a **higher computational cost**, which further increases as the grid size increases.

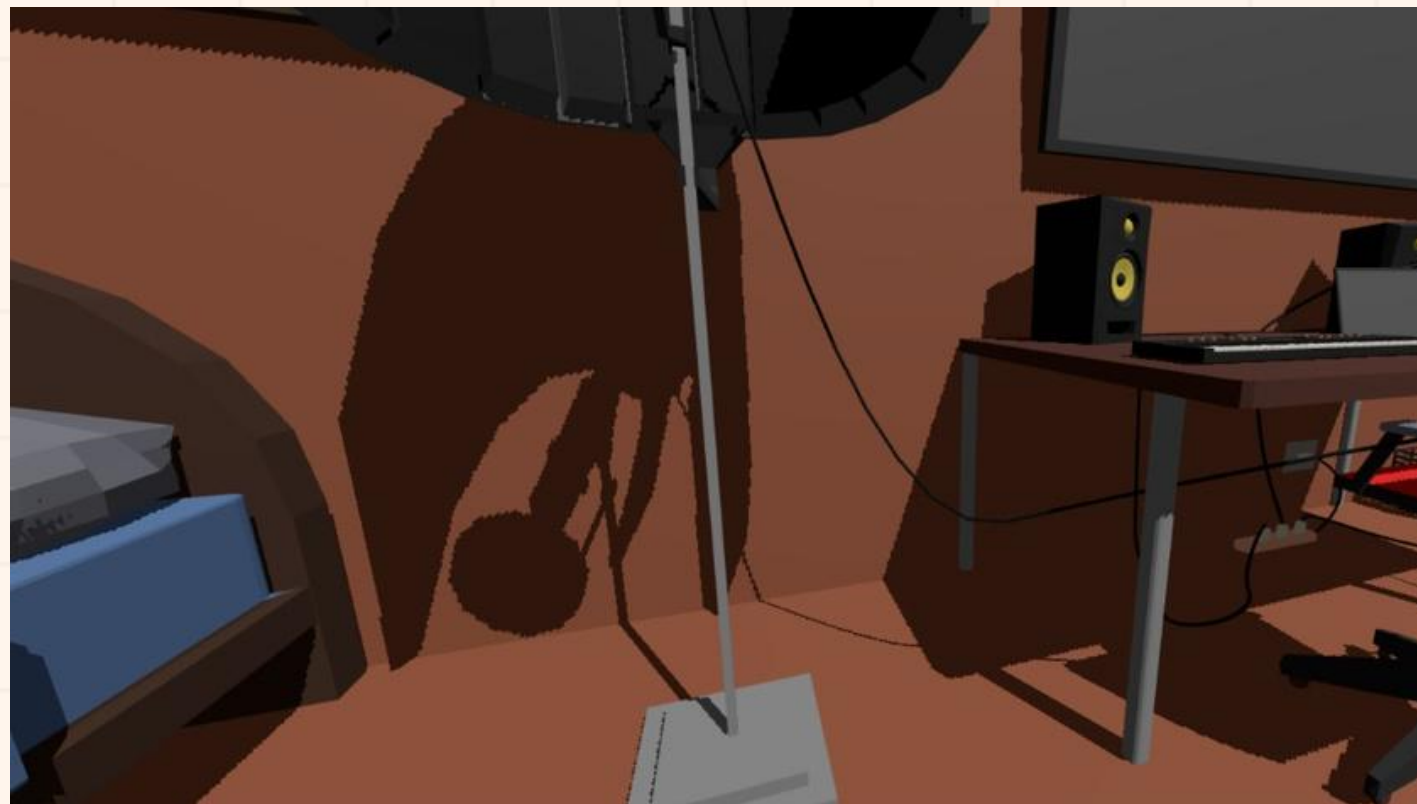
```
float getShadowPCF(vec2 uv, float depth) {  
  
    float shadow = 0.0;  
  
    float texel = 1.0 / 4096.0;  
  
    for (int x = -1; x <= 1; x++) {  
        for (int y = -1; y <= 1; y++) {  
  
            vec2 offset = vec2(float(x), float(y)) * texel;  
  
            float closest = texture2D(shadowMapCustom, uv + offset).r;  
  
            shadow += step(closest + bias, depth);  
        }  
    }  
  
    return shadow / 9.0;  
}
```




UNIVERSITÀ
DEGLI STUDI
FIRENZE

MARTINA
SCHIRONE

VISUAL EXAMPLE OF PCF AND BASIC SHADOW MAPPING



BASIC



PCF



VSM SHADOW MAPPING

Unlike Basic Shadow Mapping and PCF, this technique does not store only depth values in the shadow map.

For each texel, the first **two moments** of depth are stored:

- **Mean depth:** $E[x]$
- **Mean squared depth:** $E[x^2]$

During rendering, these values are used to compute **variance** and estimate shadowing using **Chebyshev's inequality**.

This approach introduces **additional computational overhead** due to variance evaluation but the results are very satisfying.

A known limitation is **light bleeding**, where light incorrectly leaks into areas that should be fully in shadow.

$$M_1 = E(x) = \int_{-\infty}^{\infty} xp(x)dx$$

$$M_2 = E(x^2) = \int_{-\infty}^{\infty} x^2 p(x)dx$$

$$\mu = E(x) = M_1$$

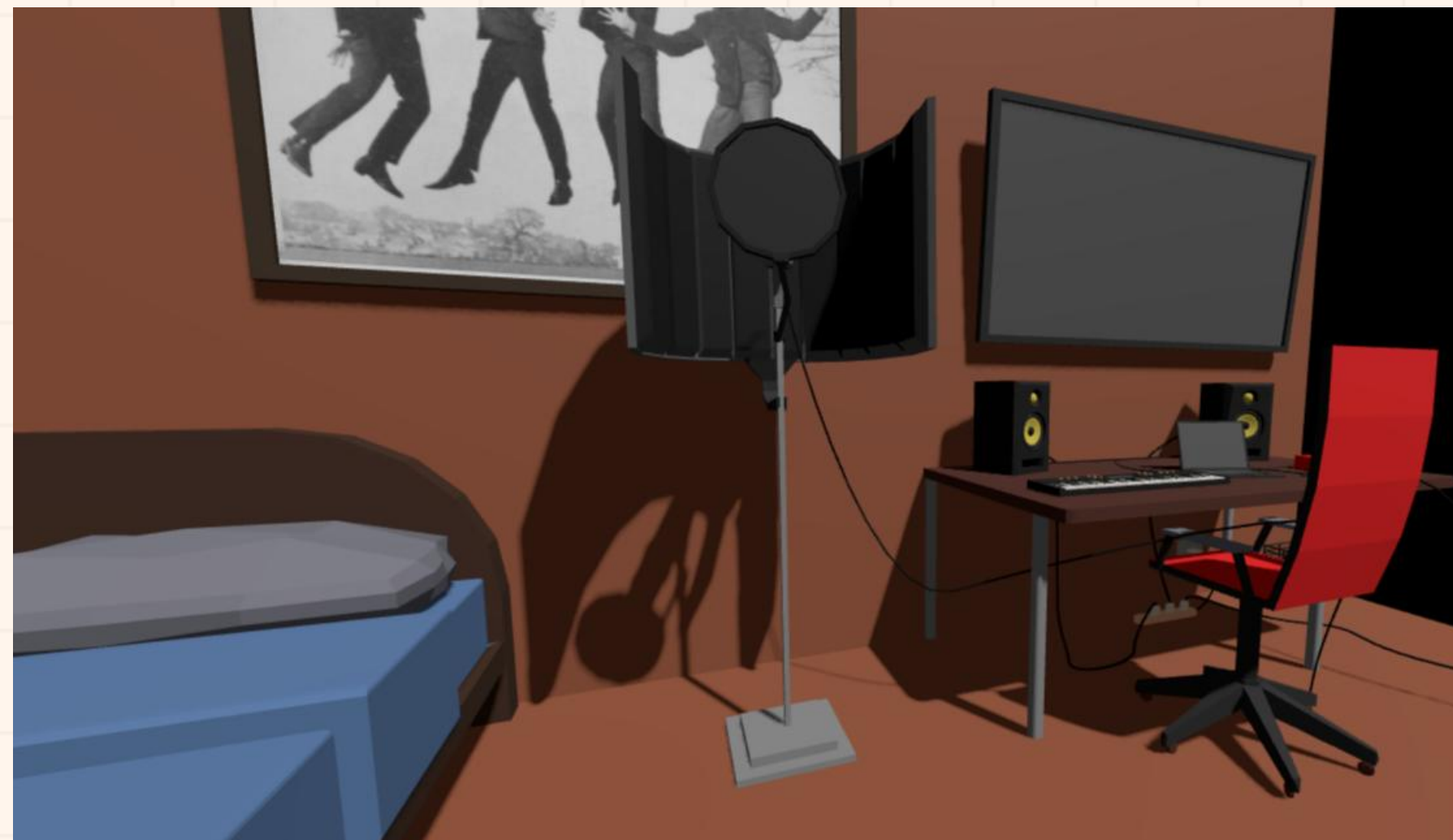
$$\sigma^2 = E(x^2) - E(x)^2 = M_2 - M_1^2$$

$$P(x \geq t) \leq p_{max}(t) \equiv \frac{\sigma^2}{\sigma^2 + (t - \mu)^2}$$



VSM SHADER AND VISUAL EXAMPLE

```
float chebyshevUpperBound(vec2 moments, float t) {  
  
    float mean = moments.x;  
    float mean2 = moments.y;  
  
    float variance = max(mean2 - mean * mean, 0.00002);  
  
    float d = t - mean;  
  
    float p = variance / (variance + d * d);  
  
    return clamp(p, 0.0, 1.0);  
}  
  
float getVSMShadow(vec2 uv, float depth) {  
  
    vec2 moments = texture2D(shadowMapCustom, uv).xy;  
  
    float t = depth - bias;  
  
    float p = chebyshevUpperBound(moments, t);  
  
    // light bleeding fix  
    return clamp(max(p, step(depth, moments.x)), 0.0, 1.0);  
}
```





SHADOWMAPPING TECHNIQUES COMPARISON OVERVIEW

Basic Shadow Mapping is fast but produces aliased shadows. PCF provides a balanced trade-off between performance and quality. VSM delivers the best visual quality but at a higher computational cost.

TYPE	PROS	CONS	FPS
BASIC	Very light to compute	Jagged edges, shadow acne	60
PCF	Smooth shadows	Medium cost	44-55
VSM	Very smooth shadows	Light bleeding, higher cost	35-38



UNIVERSITÀ
DEGLI STUDI
FIRENZE

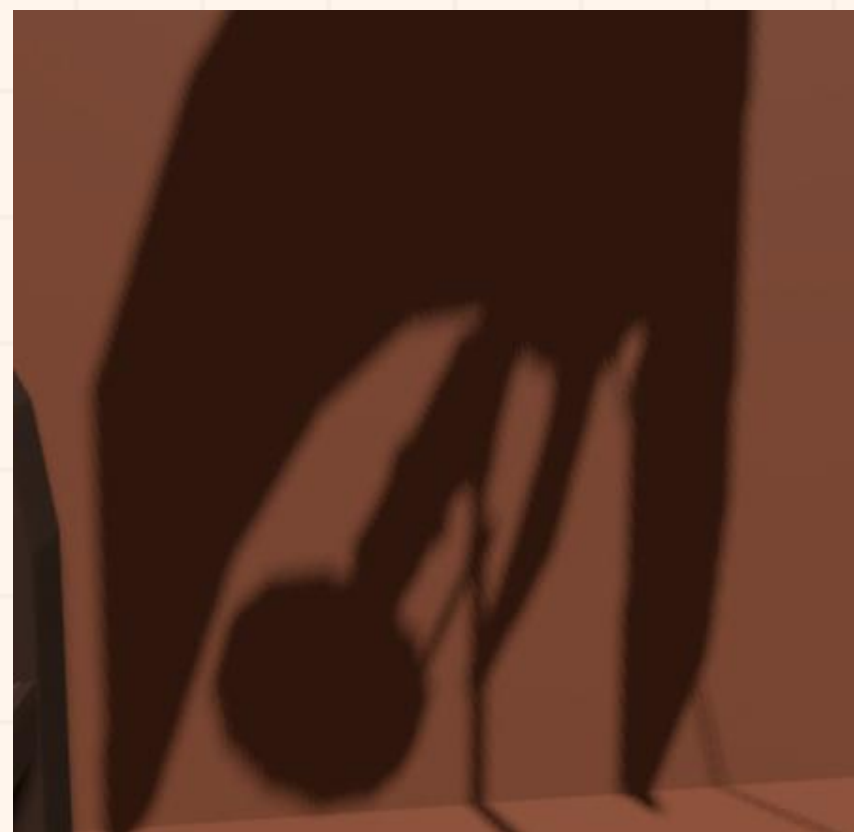
MARTINA
SCHIRONE

SHADOWMAPPING TECHNIQUES

COMPARISON OVERVIEW



BASIC



PCF

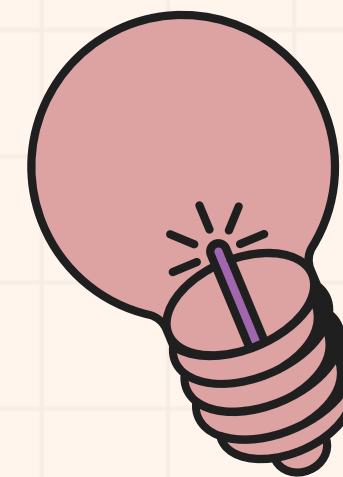
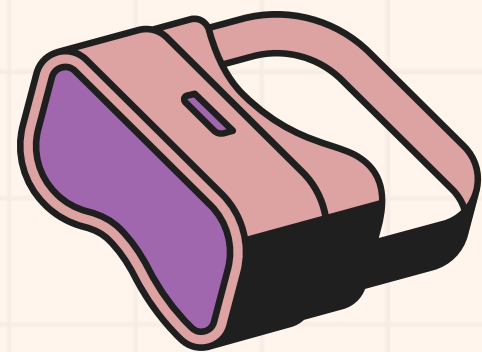


VSM



UNIVERSITÀ
DEGLI STUDI
FIRENZE

MARTINA
SCHIRONE



THANK YOU

